Contrats des 5 agents : guide pour l'équipe

Propriétaire des contrats : **Oumaima** (cheffe de projet). Toute demande de modification passe par elle.

1. À quoi servent les schémas et les mocks

Élément	Où	Rôle
Schémas Pydantic	`agents/schemas.py`	Décrivent la forme exacte de ce que chaque agent **renvoie**. Python refuse toute réponse qui ne les respecte pas. Ils servent de **contrat** entre les agents : l'agent 4 sait que `score_coherence` existera toujours, même si l'agent 3 n'est pas fini.
Mocks JSON	`samples/`	Des **exemples de sorties valides**, écrits à la main. Chacun peut tester son travail sans attendre celui des autres. L'orchestrateur de Fatima les utilise comme faux agents.
check_mocks.py	`agents/`	Vérifie que chaque mock respecte son schéma **et** que les mocks racontent la même histoire.

Le schéma d'un agent sert à trois choses : forcer le format du JSON demandé au modèle, valider sa réponse (avec relance si elle est invalide), et documenter le rapport technique.

2. Vue d'ensemble

```
Agent 1 (offre)  ─┐
                  ├─> Agent 3 (documents) -> Agent 4 (matching) -> Agent 5 (rapport)
Agent 2 (CV)     ─┘
```

Agent	Responsable	Entrée	Sortie (schéma)	Mock
1 Job Requirement	Zineb	Texte de l'offre	`ExigencesStructurees`	`agent1_exemple.json`
2 CV Extraction	Salah	Texte du CV	`ProfilCandidat`	`agent2_exemple.json`
3 Document Verification	Oumaima	Profil du CV + diplôme + CIN	`VerificationDocuments`	`agent3_exemple_conforme.json`, `agent3_exemple_incoherent.json`
4 Matching	Abdo	Sorties des agents 1, 2, 3	`ResultatMatching`	`agent4_exemple_conforme.json`, `agent4_exemple_incoherent.json`
5 Report	Fatima (prompt) et Amina (assemblage)	Sorties des agents 1 à 4	`RapportCandidature`	`agent5_exemple_conforme.json`, `agent5_exemple_incoherent.json`

Tous les exemples racontent la même histoire : l'offre « Développeur Python » et la candidate fictive Sara Benali. Les agents 3, 4 et 5 ont deux versions : une chaîne **conforme** (score 85,3) et une chaîne **incohérente** (score 71,3, cas exigé dans le jeu de test).

3. Les champs, agent par agent

Agent 1, `ExigencesStructurees` : `titre_poste`, `diplome_demande` (texte), `niveau_diplome_min` (nombre : 5 pour Bac+5), `experience_min_annees`, `competences_obligatoires`, `competences_souhaitees`, `langues` (langue + niveau).

Agent 2, `ProfilCandidat` : `nom`, `prenom`, `email`, `telephone`, `formation` (diplôme, établissement, année), `experiences` (poste, entreprise, durée), `competences`, `certifications`, `langues`, `experience_totale_annees`. Le genre, l'âge et la photo sont **volontairement absents** (bonus anti-biais).

Agent 3, `VerificationDocuments` : `infos_diplome`, `infos_cin`, `incoherences` (type, description, gravité, documents concernés), `niveau_conformite`, `score_coherence` (0 à 100), `resume`.

Agent 4, `ResultatMatching` : `score_global`, `detail` (6 critères avec pondération, score et justification), `competences_correspondantes`, `competences_manquantes`, `diplome_conforme`, `experience_candidat_annees`.

Agent 5, `RapportCandidature` : `resume`, `score_global`, `points_forts`, `manques`, `incoherences`, `justification_score`, `conclusion`, `avis_indicatif`,

`mention_decision_humaine`.

4. Les règles de cohérence (déjà vérifiées par le code)

- **Le score global de l'agent 4 est la somme pondérée des 6 critères.** Pondérations : compétences obligatoires 40 %, expérience 20 %, diplôme 15 %, compétences souhaitées 10 %, langues 5 %, cohérence des documents 10 %. Sinon Pydantic refuse la sortie.
- **Le critère « cohérence » de l'agent 4 reprend `score_coherence` de l'agent 3.**
- **Le critère « diplôme » se base sur le diplôme vérifié** (agent 3), pas sur ce que le CV déclare.
- `score_coherence` = 100 moins 40 / 20 / 10 par incohérence élevée / moyenne / faible (plancher 0). Il est calculé par le **code**, pas par le modèle.
- **Le rapport (agent 5) reprend exactement** le score et le détail de l'agent 4.

5. Ce que fait le modèle, ce que fait le code

Modèle de tous les agents : `qwen3.5:4b` (Ollama, en local). Un petit modèle lit et extrait bien, mais compare et calcule mal : ces deux tâches sont donc confiées au code.

Agent	Le modèle fait	Le code fait
1	Extraire les exigences de l'offre	Valider le JSON
2	Extraire le profil du CV	Valider le JSON, recalculer l'expérience totale
3	Lire les images, extraire les champs	Comparer avec le CV, calculer la conformité
4	Reconnaître les compétences équivalentes (JS = JavaScript)	Calculer le score pondéré
5	Rédiger le résumé et la conclusion	Assembler points forts, manques, incohérences, justification

6. Comment utiliser les contrats dans votre code

```python
from schemas import ExigencesStructurees
import json

# valider un JSON (mock ou sortie d'agent)
exigences = ExigencesStructurees.model_validate_json(texte_json)

# récupérer un dictionnaire pour la base de données
donnees = exigences.model_dump()
```

Pour vérifier que tout est en ordre à tout moment : `python check_mocks.py` (dossier `agents/`).

7. Règles de l'équipe

1. **Ne modifiez jamais** `schemas.py` ou un mock vous-même.
2. Si votre agent a besoin d'un champ en plus ou en moins : écrivez un commentaire sur la tâche ClickUp « Figer les schémas JSON/Pydantic » en expliquant **pourquoi**. Oumaima décide, puis prévient tout le monde.
3. Après tout changement : relancer `python check_mocks.py` et les tests de votre agent.
4. Les champs que votre agent **reçoit** viennent du schéma de l'agent précédent : utilisez les mêmes noms.

8. Ce que chaque responsable doit vérifier pendant la relecture

- **Zineb (agent 1)** : toutes les exigences d'une offre se retrouvent dans les champs ? Un champ manque-t-il ?
- **Salah (agent 2)** : le profil contient-il tout ce qu'un CV peut donner ? Les champs sont-ils extractibles par un petit modèle ?
- **Oumaima (agent 3)** : les entrées de l'agent 3 correspondent-elles aux champs de l'agent 2 (`nom`, `prenom`, `formation`) ?
- **Abdo (agent 4)** : a-t-il toutes les entrées nécessaires au calcul des 6 critères ? La règle de somme pondérée lui convient-elle ?
- **Fatima et Amina (agent 5)** : le rapport contient-il tout ce que la fiche demande (points forts, manques, incohérences, conclusion) ?

Date limite de relecture : **fin du jour 2**. Après la validation, les contrats sont gelés.